



Chapter 9 — Implementation Strategy & Phase Roadmap

9.1 Strategy Overview

DOIP is built in **7 phases across 4 layers**, exactly as the critical-perspective analysis recommends:

Layer	Phases	Outcome
Core platform	1–2	Map, simulation, events, missions — a usable system early
AI layer	3–4	Summarization, RAG assistant, computer vision
Analytics layer	5–6	Logistics, health, weather, predictions
Advanced integration	7	Kafka, Kubernetes, monitoring, CI/CD hardening

Working rules

1. **Never start a phase until the previous phase's exit criteria pass.** Half-finished modules are the #1 failure mode for a project of this size.
2. Every phase ends with a **tagged release + 2–3 minute demo recording + changelog**. These become milestones you can showcase at any time.
3. Timebox: if a feature exceeds its sprint budget by 50%, cut scope (documented in the changelog), don't extend.
4. Estimated effort assumes ~10–15 focused hours/week by a solo developer. Total: **~7–9 months**. With more hours or teammates, compress proportionally.

9.2 Phase 0 — Foundations (Week 0–1)

Repository setup before any features:

- Monorepo scaffold (structure in Chapter 12), pre-commit hooks, Ruff + mypy + ESLint, `.env.example`.

- `docker compose up` brings up Postgres+PostGIS+pgvector, Redis, MinIO, backend stub, frontend stub.
- CI pipeline: lint → typecheck → test on every push.
- Alembic baseline migration; seed script skeleton; `make dev / test / seed` targets.

Exit criteria: a new machine goes from `git clone` to running stack in < 10 minutes.

9.3 Phase 1 — Interactive Map + Simulated Units & Incidents (Weeks 2–7)

Sprint	Deliverables
1.1	Auth (JWT login, roles), users table, protected routes, app shell with dark ops theme
1.2	Simulation engine v1: tick loop, world state, seeded RNG, <code>UnitSpawned</code> / <code>UnitMoved</code> events; scenario YAML parser (entities + timeline only)
1.3	WebSocket gateway: subscribe/resume protocol, snapshot endpoint, Redis pub/sub wiring
1.4	MapLibre map: OSM tiles, unit markers with smooth interpolation, layer toggles, entity side panel
1.5	Incidents: scenario-scripted spawning, markers, event feed widget, acknowledge/resolve actions
1.6	PostGIS showcase: “units within 5 km of incident” panel, sector zones rendering; polish + demo recording

Exit criteria (the Phase-1 bar from the project brief): 5–10 units moving live via WebSocket; incidents spawn from scenario files with detail panel; live event feed; at least one real geospatial query in the UI; clean restart recovers state.

9.4 Phase 2 — Mission Planning & Route Optimization (Weeks 8–12)

Sprint	Deliverables
2.1	Draw tools (zones, waypoints); mission CRUD; restricted zones as hard constraints

Sprint	Deliverables
2.2	Routing graph over operational area; Dijkstra baseline , then AI • with terrain/weather/risk cost function; route-vs-naive comparison view
2.3	Coverage path generation (lawnmower) for search zones; battery & duration estimator with feasibility verdict + explanations
2.4	Mission execution: launch → engine flies route → WaypointReached / MissionCompleted events; mission timeline widget; replay slider v1

Exit criteria: optimized route visibly avoids restricted polygon and beats naive cost; infeasible mission explains why; launched mission runs live on the map.

9.5 Phase 3 — AI Report Summarization + RAG Assistant (Weeks 13–18)

Sprint	Deliverables
3.1	AI service scaffold: Ollama integration, provider interface, SSE streaming chat panel
3.2	Situation Reporter pipeline (LangGraph): collect → cluster → draft → grounding verification → store; auto + on-demand generation; sitrep timeline UI
3.3	RAG ingestion: author 10–15 starter SOP documents; chunk/embed into pgvector; ingestion API
3.4	RAG query pipeline with hybrid retrieval + citations; router (chat vs RAG); golden-set eval harness (make eval-ai)

Exit criteria: sitrep meets the FR-3 acceptance test (20 events → correct prioritization, zero hallucinated facts); RAG answers 80%+ of golden set with correct source citations.

9.6 Phase 4 — Computer Vision on Prerecorded Footage (Weeks 19–23)

Sprint	Deliverables
4.1	Vision service extraction (first real microservice): Celery worker, MinIO storage, job API with progress
4.2	YOLO pipeline: frame sampling → inference → ByteTrack dedupe → detections table; sample public footage curated

Sprint	Deliverables
4.3	Risk scoring + auto-incident creation; map plotting of detections
4.4	Reviewer UI (annotated filmstrip, confirm/dismiss); optional fine-tune on aerial dataset; demo

Exit criteria: FR-5 acceptance test — 2-minute clip processes end-to-end into reviewable, risk-ranked, map-visible detections.

9.7 Phase 5 — Logistics & Personnel Monitoring (Weeks 24–28)

Sprint	Deliverables
5.1	Logistics engine subsystem: activity-driven consumption, depot/unit inventories, threshold alerts
5.2	Logistics dashboard: levels, burn-rate charts, time-to-empty projections (extrapolation mode); resupply missions via planner
5.3	Vitals subsystem: synthetic generator (baseline + circadian + activity + anomaly injection); roster UI with sparklines
5.4	Health alerting (threshold + trend), map/feed integration, scenario hooks

Exit criteria: moving units burn fuel visibly; a scripted shortage raises early warnings; injected heat-exhaustion anomaly triggers a health alert end-to-end.

9.8 Phase 6 — Predictive Analytics & Weather Integration (Weeks 29–34)

Sprint	Deliverables
6.1	Weather subsystem: moving cells, intensity evolution, map overlays, 6h forecast panel
6.2	Operational coupling: speed/route-cost/feasibility modifiers with visible multipliers (FR-9.3)
6.3	Analytics service extraction; feature aggregation pipeline; Prophet + XGBoost consumption & ETA models with uncertainty bands
6.4	After-action report page (charts + AI narrative); model registry (MLflow); forecast lineage

Exit criteria: storm demo — weather cell slows a unit, reroutes a mission, shifts a forecast; after-action report generates for any completed run.

9.9 Phase 7 — Production Architecture: Kafka, Kubernetes, CI/CD, Monitoring (Weeks 35–40)

Sprint	Deliverables
7.1	Redis → Kafka migration: topic design, consumer groups, dual-write cutover, replay tooling; written migration ADR
7.2	Compose → k3s: manifests/Helm chart, Ingress + TLS via Nginx, secrets management, HPA for vision workers
7.3	Observability: Prometheus metrics (tick duration, WS clients, event lag, AI latency), Grafana dashboards, alert rules, Loki logs
7.4	CI/CD completion: image build & push, automated deploy, load test (200 entities / 50 events/s), security pass (rate limits, headers, audit review), final demo film

Exit criteria: one-command K8s deploy; Grafana shows live platform health during a run; load test meets NFR-1/NFR-2; full 3-minute showcase demo recorded.

9.10 Risk-Managed Sequencing — What to Cut If Time Runs Short

Ordered fallback plan (cut from the bottom):

1. Keep: Phases 1–3 (core + AI) — this alone is a complete, impressive product.
2. Cut first: LSTM experiments (keep Prophet/XGBoost), Loki, personnel health module.
3. Cut second: Kubernetes (ship on Compose + document the K8s plan), CV fine-tuning (ship pretrained classes only).
4. Never cut: simulation determinism, WebSocket resume, sitrep grounding verification — these are the platform's credibility.

9.11 Definition of Done (applies to every feature)

- ☐ Typed API contract + OpenAPI docs updated
- ☐ Unit tests + one integration test; golden-run suite still green
- ☐ Works through a full pause/resume/restart cycle

- ☐ Emits/consumes events through the standard schema only
- ☐ Dashboard/UI state handles loading, error, and empty states
- ☐ Changelog entry + screenshot/GIF for the phase demo reel